WebRTC Voice on Second Life, a Developer's View

This project details moving Second Life voice to WebRTC (Web Real-time Communications.)

NOTE: This is a living document and will be updated as questions arise.

Background

Second Life currently provides four types of voice communication:

- Spatialized voice, for in-world communication.
- Group voice.
- Peer to peer voice.
- Ad-Hoc voice

We currently use a 3rd party provider (Vivox) to provide voice-as-a-service. Vivox has been the primary voice provider since voice was added to the product over a decade ago.

It's time for an update.

WebRTC (Web Real-Time Communications) is the predominant telephony protocol used by web-based applications, such as Google Meet. It's built-in to Chrome, Safari, Firefox, and many other web browsers, and allows audio communication as well as data and video. All of those browsers support an open API allowing access to WebRTC functionality via javascript.
The core implementation of WebRTC is an open source C++ package, actively maintained by Google, as well as many others. (https://webrtc.org/)

Features provided by the webrtc.org implementation include:

- NAT hole punching via STUN.
- Data relaying via TURN.
- Audio, Video, and Data transmission.
- Audio/video device selection.

- Stereo audio.
- Configurable audio/video bandwidth allowing control over audio and video quality.
- Audio noise reduction.
- Audio automatic gain control.
- Audio echo cancellation
- Multiple audio tracks per stream.
- Multiple streams per connection.
- Simultaneous connections to multiple WebRTC sources (servers, clients, etc.)
- Peer 2 peer connection (directly or via relay)
- Communication with peers via servers (SFU, MCU)
- Improve privacy and security.
- And many more.

We're moving over to self-hosted WebRTC-based voice to gain those features, and to open the door for future features.

As the voice subsystem in the viewer is modular, the major changes are limited to a new WebRTC voice module and a supporting dynamic library which wraps the native webrtc library.

Changes from Vivox include:

- Spatial voice for a given region is handled by a Second Life Voice Server running in conjunction with that region.
- AdHoc and Group voice is handled by a pool of Second Life Voice Servers dedicated to those types of voice.
- Unlike Vivox's direct-call mechanism, WebRTC P2P is routed through the AdHoc voice server pool as if it were a two-person Ad-Hoc voice conference.
- There is no separate SLVoice executable for WebRTC Voice (in the short term, we'll allow both WebRTC Voice and Vivox voice to coincide, so SLVoice will be running to provide Vivox support.)

WebRTC Voice Interface

The following documentation describes the caps and data channel interface to do webrtc voice on WebRTC spatial voice enabled regions and on environments that allow WebRTC P2P/Ad-Hoc/Group voice.

It’s assumed that the reader has some understanding of WebRTC signaling.

Caps / Peer Connection Provisioning

Second Life WebRTC Voice uses two capabilities, communicated to the simulator from the viewer (desktop or mobile.)
These two caps apply to both Spatial and P2P/Ad-Hoc/Group voice.

ProvisionVoiceAccountRequest

The ProvisionVoiceAccountRequest cap takes a post with either of the two following payloads, in LLSD (as CAPS generally are.)

Spatial Voice

To connect to a voice session for a region (or parcel,) the client sends the following:

```
<llsd>
   <map>
      <key>jsep</key>
      <map>
         <key>type</key>
         <string>offer</key>
         <key>sdp</key>
         <string>....sdp content…</string>
       </map>
       <key>parcel_local_id</key>
       <integer>2</integer>
       <key>channel_type</key>
       <string>local</string>
       <key>voice_server_type</key>
       <string>webrtc</string>
   <map>
</llsd>
```

The JSEP (Javascript Session Establishment Protocol) contains the information needed to negotiate a WebRTC connection between the client and the Second Life Voice Server. It's an "offer" in WebRTC terminology.
The **parcel_local_id** is an optional integer parcel identifier, scoped to within the region.
The **channel_type** is either **local** for spatial connections, or **multiagent** for adhoc/p2p/group connections.
voice_server_type should only be **webrtc**.

The above LLSD is passed to the simulator, which then negotiates with the Second Life Voice Server to retrieve an answer sdp, which is passed back as LLSD as follows.

```
<llsd>
  <map>
    <key>jsep</key>
    <map>
      <key>type</key>
      <string>answer</key>
      <key>sdp</key>
      <string>....sdp content…</string>
    </map>
    <key>viewer_session</key>
    <string>...viewer session…</string>
  </map>
</llsd>
```

The offer should be retrieved from webrtc when creating a peer connection, and should be mangled so that the **a=fmtp**: line should have a value of **minptime=10;useinbandfec=1;stereo=1;sprop-stereo=1;maxplaybackrate=48000**

More information about the SDPs in the offer and answer can be found here.

Connections are closed by sending the following LLSD to ProvisionVoiceAccountRequest:

```
<llsd>
  <map>
    <key>logout</key>
    <boolean>true</boolean>
    <key>voice_server_type</key>
    <string>webrtc</string>
    <key>viewer_session</key>
    <string>...viewer session…</string>
  </map>
</map>
```

NOTE: The CAP is named “ProvisionVoiceAccountRequest” for historical reasons, as Vivox is set up to create a voice account attached to a voice channel using sip. Technically, webrtc is simply creating a voice channel associated with the Second Life account.

The ‘logout’ request may return 404 if the connection has already been closed through other means, such as exit of the avatar, shutdown of the webrtc connection, etc.

AdHoc/Group/P2P Voice

Multiagent (AdHoc/Group/P2P) voice uses a pool of non-spatizing Second Life Voice Servers to establish voice sessions between two or more people. The chat subsystem in Second Life negotiates a call which results in information needed to connect to the session, such as channel ID, credentials, voice server type, etc.

This information is passed in via the ProvisionVoiceAccountRequest in the form of:
channel - channel id
credentials - channel credentials.

```
<llsd>
   <map>
      <key>jsep</key>
      <map>
         <key>type</key>
         <string>offer</key>
         <key>sdp</key>
         <string>....sdp content…</string>
       </map>
       <key>channel</key>
       <string>...channel id…</string>
       <key>credentials</key>
       <string>...channel credentials…</string>
       <key>channel_type</key>
       <string>multiagent</string>
       <key>voice_server_type</key>
       <string>webrtc</string>
   <map>
</llsd>
```

The connection is closed via the same closing mechanism used for Spatial Voice above.

VoiceSignalingRequest

The other cap used in WebRTC spatial voice is the VoiceSignalingRequest cap. It is used solely for trickling ICE candidates after the session has been established.

It's a POST request that takes the following LLSD for new ice candidates:

```
<llsd>
   <map>
      <key>candidates</key>
      <array>
         <map>
            <key>sdpMid</key>
            <string>sdp mid value</string>
            <key>sdpMLineIndex</key>
            <integer>index</integer>
            <key>candidate</key>
            <string>ice candidate string</string>
```

```
        </map>
        <map>
          <key>sdpMid</key>
          <string>sdp mid value</string>
          <key>sdpMLineIndex</key>
          <integer>index</integer>
          <key>candidate</key>
          <string>ice candidate string</string>
        </map>
        …
      </array>
      <key>voice_server_type</key>
      <string>webrtc</string>
      <key>viewer_session</key>
      <string>...viewer session…</key>
    </map>
</llsd>
```

Each ICE candidate is defined by a candidate string detailed here,

It's expected that one or more of these ice candidate POST calls will be made AFTER the answer is received from the ProvisionVoiceAccountRequest offer. Any ice candidates that are gathered before the answer response is received should be cached and sent after the response is received.

Signaling ICE candidates after the answer has been received is known as ICE Trickling.

When gathering is complete, the following should be sent via the VoiceSignalingRequest cap.

```
<llsd>
  <map>
    <key>candidate</key>
    <map>
      <key>completed</key>
      <boolean>true</boolean>
    </map>
    <key>voice_server_type</key>
    <string>webrtc</string>
    <key>viewer_session</key>
    <string>...viewer session…</key>
  </map>
</llsd>
```

Data Channel Interface for WebRTC Voice

The server listens for and sends data on the "**SLData"** data channel. Clients must create the "**SLData**" data channel before negotiation, just as clients create the audio channels.

Communication over the data channels consists of json as strings (as opposed to binary.)

Client -> Mixer

The base json object sent to the server can have any of the following members:
"j" : join (an object) - indicating that this client is joining a session. The sole member of the object is an option "p" which indicates this is a primary connection. Primary connections are the ones for which audio levels are streamed back to the client. The default for "p" is false.
"l": leave (a boolean, always true) - indicates that this client is leaving a session.
"sp" : sender position (an object) containing integer "x," "y," and "z," coordinates for the sender. These coordinates are world coordinates (not region-relative) in centimeters. (only for spatial voice)
"sh" : sender heading (an object) containing integer "x," "y," "z," "w" quaternion values (multiplied by 100) (only for spatial voice)
"lp" : listener position (an object) containing integer "x," "y," and "z," coordinates for the sender. These coordinates are world coordinates (not region-relative) in centimeters. (only for spatial voice)
"lh" : listener heading (an object) containing integer "x," "y," "z," "w" quaternion (multiplied by 100). (only for spatial voice)
"m": agents to mute or unmute. An object associating agent ids with true/false (mute/unmute)
"ug": set gain on agents. An object associating agent ids with integer gains (gain * 200)

All values are not required for an update. It's expected that values should only be sent if they're changed, to reduce bandwidth.

Example:

```
{
   "j": { "p": true },
   "sp": { "x": 37000, "y" : 2400, "z" : 37201 },
   "lh" : { "x": 100, "y" : 50, "z": 1, "w": 100 },
   "m" : {
            "98d0084e-a6eb-45ce-b847-4da34ea823a8":true,
            "95c8ef1a-edcb-467c-88fe-ea503e5b974e": false
           },
   "ug": {
            "98d0084e-a6eb-45ce-b847-4da34ea823a8":27,
            "95c8ef1a-edcb-467c-88fe-ea503e5b974e": 125
           },
}
```

Mixer->Client
The mixer will send json to each client containing an object, with each member being an object indexed by the agent id of the peer.

The following values will show up on those per-peer objects:
“p” : power level (an integer) which is RMS * 128 (for the individual VU meters)
“V” : VAD (voice activity detected.) The value will be true if the peer is talking.
“j” : join (an object) - indicating that this client is joining a session. The sole member of the object is an option “p” which indicates this is a primary connection. Primary connections are the ones for which audio levels are streamed back to the client.
“l”: leave (a boolean, always true) - indicates that this peer has left a session.

An example is as follows:

```
{
    “98d0084e-a6eb-45ce-b847-4da34ea823a8” : { “p” : -43, “v”: true },
    “b27a917b-fc3a-4bf4-bb50-e179d23e47c4” : {“j”: { “p”: false } },
    “95c8ef1a-edcb-467c-88fe-ea503e5b974e” : {“l”: true}
}
```

These values and notifications will be batched and sent to the client at a relatively snappy pace (100ms)

Cross-Region Spatial Voice

To handle voice across a region boundary, the client is expected to make a connection both to its region and to neighboring regions that are within the audio range. The client should then send position updates to all connected regions. The client should play audio from all connected regions, but should only send audio to its primary region, with audio sent to other regions being disabled.
To assist the client in managing this, the server implements the concept of “primary” regions. When a client joins its own resident region, it should mark it as primary with “p” being true in its “j” (join) message over the datachannel. Other regions should be marked with “j->p” being false. The server will enforce audio streaming to/from regions as appropriate depending on the “primary” setting, but the client is expected to as well to reduce audio bandwidth.

Teleporting

When teleporting to another region, or crossing a region boundary, the client should first determine if it already has a connection open to that region (the of neighboring regions.) If so,

the client can simply disable audio going from the client to its old region, and enable outgoing audio for the new region.
If the client does not have a connection to the new region, it should establish one, and shut down connection to the old region.

One algorithm that can be used to determine the neighboring region and teleport behavior is:

1) Determine the list of nearby regions.
2) Connect to any of those regions not currently connected to.
3) Disconnect from any connected regions not on that list.

Parcel Voice

When crossing into a parcel with voice enabled, but limited to only itself, all other connections should be shut down, and a new connection should be established to that parcel.

When the parcel has voice disabled, all connections should be shut down, or minimally disabled.